

GIBS Interactive Build Checklist

A working checklist for instructional designers and developers building HTML learning interactives for Blackboard Ultra. Covers the GIBS Interactive Standard, accessibility requirements, common anti-patterns, and the design principles behind the standard.

Use this before, during and after building each interactive. If every box is ticked, the interactive should land in Blackboard without rework.

1. Before you start: shape of the file

- ☐ One self-contained `.html` file. No external CSS files, no external JavaScript files, no build step.
 - ☐ One `<style>` block scoped under the `.gibs-interactive` namespace (or a unique component ID like `#gibs-team-process`). No global selectors that target `body`, `*`, `h1`, `button`, etc.
 - ☐ No external font imports. Use the system font stack: `system-ui, -apple-system, "Segoe UI", Roboto, Helvetica, Arial, sans-serif`. Do not use Google Fonts, Adobe Fonts or any CDN font.
 - ☐ No external image, icon or library imports. SVG icons inline, no FontAwesome, no jQuery.
 - ☐ One `<script>` block at the bottom of the file, wrapped in an IIFE `((function(){ ... }()));` to keep variables out of the global scope.
-

2. The hero pattern (every interactive)

Every interactive opens with the same panel:

- ☐ Navy background (`--blue: #002c77`), white text, rounded corners, soft shadow.
- ☐ One `<h1>` with the interactive title.
- ☐ One `<p>` immediately below with a one-sentence instruction (e.g. "Click each component to reveal its description.").
- ☐ Centred text alignment.
- ☐ No emoji icons in headings.

This is your visual signature. If two interactives in the same course have different hero patterns, learners notice the inconsistency before they notice anything else.

3. Data over markup (the most important rule)

If the interactive shows multiple items that share the same structure (cards, accordion items, slides, tabs, equations, segments of a wheel), define those items as a JavaScript array of objects and render them with one template. Do **not** hand-code each item.

- ☐ All repeated content lives in a single JS data array near the bottom of the script.
- ☐ One render function builds the markup from that array.
- ☐ Adding a new item is a one-line addition to the array, nothing more.
- ☐ If you find yourself copying and pasting an HTML block, stop and lift it into the data array first.

Why it matters: when you hand-code seven slides or four accordion items, every accessibility fix, every style tweak, every typo correction has to happen seven or four times. Data-driven means one fix, everywhere.

4. Accessibility (non-negotiable)

Keyboard

- ☐ Every interactive element is reachable with the Tab key.
 - ☐ Every button is a real `<button type="button">`. Never use `<div>` or `<span>` with click handlers.
 - ☐ Visible focus rings: `:focus-visible` outline in teal (`--teal: #007378`), 3px, with 3px offset.
 - ☐ **Accordion:** Down/Up arrows move focus between headers, Home and End jump to first/last.
 - ☐ **Tabs:** Left/Right (or Up/Down) arrows move between tabs, Home and End jump to first/last.
- Roving `tabindex` so only the active tab is in the page tab order.
- ☐ **Slide deck:** Left/Right arrow keys move between slides, Home and End jump to first/last.
 - ☐ A small "Tip:" line below the controls tells learners which keys work.

Screen readers

- ☐ Skip link at the top: `<a class="gibs-skip" href="#main-id">Skip to main content</a>`. Hidden until focused.
- ☐ One `<main>` element wraps the interactive content (not `role="main"` on a div).
- ☐ Heading hierarchy: one `<h1>` (page title), `<h2>` for major sections, `<h3>` for sub-items if needed. Never skip levels.
- ☐ Buttons that toggle content use `aria-expanded="true"` / `"false"` and `aria-controls="panel-id"`.
- ☐ The panel they control has the matching `id` and `role="region"` with `aria-labelledby="button-id"`.
- ☐ Tabs use `role="tab"` inside a `role="tablist"`, and panels use `role="tabpanel"`.

- ☐ Content that updates (e.g. revealing a card or switching a tab) lives inside an `aria-live="polite"` region so changes are announced.
- ☐ Decorative icons have `aria-hidden="true"`. Never put `aria-hidden` on a focusable element.
- ☐ SVG diagrams have `<title>` and `<desc>` elements, plus a group `aria-label`.

Reduced motion

- ☐ `@media (prefers-reduced-motion: reduce)` block disables all transitions and transforms.

Forms

- ☐ Every input has a `<label>` (visible or visually hidden via `.sr-only`), never just a placeholder).
-

5. Visual standards

Palette

Use only these brand variables. Define them once at the top of the namespace.

```
--blue:    #002c77;    /* Primary, hero, default buttons */
--teal:    #007378;    /* Focus rings, accents */
--purple:   #5c2f6f;    /* Accent */
--orange:   #C9632f;    /* Accent, calls to action */
--maroon:   #7B293D;    /* Accent */
--grey:     #6b7280;    /* Neutral accent */
--ink:      #000000;    /* Body text */
--muted:    #4f5668;    /* Secondary text */
```

- ☐ No colours outside this palette except greys for borders and backgrounds.
- ☐ If colour-coding categories or steps, assign one brand colour per category and use it consistently across the visual, the heading, the border and any badges.

Typography

- ☐ System font stack only (see section 1).
- ☐ Headings use `font-weight: 800` (extra bold).
- ☐ Body text is 16px minimum. Use `clamp()` for headings so they scale on small screens.
- ☐ No `text-transform: uppercase` on long passages of text. Headings only, and sparingly.

Layout

- ☐ Maximum content width around 900-1100px, centred with `margin: 0 auto`.

- ☐ Cards and panels use `border-radius: 14-16px` and a soft shadow `0 12px 26px rgba(0,0,0,.10)`.
 - ☐ Generous padding inside panels (16-24px).
 - ☐ Responsive: collapses to one column on screens narrower than 760px. Test in the Blackboard Ultra mobile view.
 - ☐ **Never** use a fixed pixel height with `overflow: auto` to constrain content. This creates a "scroll jail" inside Blackboard's own scroll. Let content expand naturally.
-

6. Print mode (every interactive)

Every interactive must produce a clean, complete PDF when the learner clicks "Print to PDF".

- ☐ A `@media print` block.
 - ☐ All interactive content visible in print, regardless of which tab/slide/accordion item is currently open in the live view.
 - ☐ Two acceptable patterns:
 - **Pattern A:** A separate "print stack" `<div>` built from the same data array on page load, hidden in screen view, shown in print view.
 - **Pattern B:** For accordions, a `beforeprint` event handler that expands all panels.
 - ☐ `page-break-inside: avoid` on each card/slide/section so PDFs break cleanly.
 - ☐ Hero text and accent colours preserved with `print-color-adjust: exact` and `-webkit-print-color-adjust: exact`.
 - ☐ The "Print to PDF" button itself is hidden in print view.
 - ☐ Test by actually printing to PDF, not just by trusting the print preview.
-

7. Controls and actions

Every interactive should offer:

- ☐ **Reset / Collapse** button (ghost style) to return to the default state.
- ☐ **Expand all / Show all** button where the interactive has multiple revealable items, so learners can read everything in one view.
- ☐ **Print to PDF** button.
- ☐ Buttons sit in a centred flex row below the main interactive area, with consistent spacing.
- ☐ Solid buttons for primary actions (Print), ghost buttons for secondary actions (Reset, Expand, Collapse).
- ☐ Minimum touch target 44×44px (`min-height: 44px`).

8. Recurring content fixes (always check)

These come up in nearly every file. Build them into your editorial pass.

Spelling and punctuation

- ☐ UK / South African spelling throughout: organisation, analyse, behaviour, prioritise, colour.
- ☐ Curly quotes are fine, just be consistent.
- ☐ No em dashes used as a stylistic flourish. Where the source uses them, replace with a comma, full stop or "and".
- ☐ No double spaces between sentences.
- ☐ Watch for "-for example" style hyphens with no space after.

Markup validity

- ☐ `<ul>` only contains `<li>` elements as direct children. Never `<p>` directly inside `<ul>`.
- ☐ `<p>` does not contain block-level elements like lists or other paragraphs.
- ☐ Every closing tag matches an opening tag in the right order.
- ☐ No two `<body>` tags. (Yes, this happens.)

Numbering and labels

- ☐ If a graphic from the source has numbered steps, check that the numbering follows a meaningful order (cascade, sequence, narrative). If it just reflects a left-to-right grid layout, strip the numbers or relabel.
- ☐ Acronyms expanded on first use: "ALP (Action Learning Project)", then "ALP" thereafter.
- ☐ Look for duplicated words in headings ("Research Methodology Method", "Cousins' Strategic Strategic Wheel").

Source content sanity-check

- ☐ Spot-check any technical term, formula or named example. Common errors: "andcan" instead of "andon", "honor" instead of a real DMAIC step, missing space after a heading, two bullets that say the same thing.
 - ☐ If a list item is one bullet with seven comma-separated sub-items, restructure as a nested list.
 - ☐ If the title doesn't match the content (e.g. "Glossary" containing report sections), rename or restructure.
-

9. Anti-patterns (never do these)

- ❌ Checkbox-hack accordions (`<input type="checkbox">` styled with `<label>` as the toggle). Use real `<button>` elements with `aria-expanded`.
 - ❌ Inline `onclick` handlers. Use `addEventListener` in the script block.
 - ❌ External font loads (Google Fonts, Adobe Fonts, anything fetched over the network).
 - ❌ Fixed-height containers with internal scrolling. Let content flow naturally.
 - ❌ Hand-coding multiple near-identical HTML blocks instead of generating from data.
 - ❌ Loading the same library multiple times (MathJax once per page, not once per equation).
 - ❌ `display: none` on content that should still be in the print PDF. Use `hidden` for collapsed accordion panels and provide a print stack.
 - ❌ `aria-hidden="true"` on a focusable element (button, link, input).
 - ❌ Heading hierarchy that skips levels (h1 → h3 with no h2).
 - ❌ Reset buttons that auto-hide a status message after a few seconds. Let users read at their own pace.
 - ❌ Photographic stock images or AI-generated portraits to represent fictional AI personas. Use a stylised avatar plus an "AI persona" label.
 - ❌ Real names, stock photos or quotes attributed to real public figures in fictional scenarios.
 - ❌ Hand-coded duplicate content for screen view and print view. Build print from the same data.
-

10. Pre-flight check (run before handing over)

Open the finished interactive and verify:

- ☐ Tab key reaches every interactive element in a sensible order.
- ☐ Arrow keys work in any tab list, accordion or slide deck.
- ☐ Escape, Enter and Space behave as expected on buttons.
- ☐ All buttons have visible focus rings.
- ☐ Resize the browser to 360px wide. Nothing overflows, no horizontal scroll, all touch targets still tappable.
- ☐ Click "Print to PDF". Open the PDF. Every piece of content is present, in a sensible order, with no clipped panels.

- ☐ Run the page through a screen reader (VoiceOver on Mac, NVDA on Windows, TalkBack on Android) for at least 30 seconds. Verify announcements make sense.
 - ☐ Run the HTML through validator.w3.org/nu and fix any errors.
 - ☐ Open the file in Blackboard Ultra in an actual course shell, not just locally. Confirm the iframe sizing behaves and no styles bleed in or out.
 - ☐ Read every visible word out loud. Catches typos, awkward phrasing and "research methodology method".
-

Quick reference: file template

Every interactive should follow this skeleton.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>[Interactive title]</title>
  <style>
    .gibs-interactive {
      /* brand variables */
      /* component styles */
    }
    @media (prefers-reduced-motion: reduce) { /* disable transitions */ }
    @media print { /* print mode */ }
  </style>
</head>
<body>
  <section class="gibs-interactive" aria-labelledby="title">
    <a class="gibs-skip" href="#main">Skip to main content</a>
    <div class="wrap">
      <header class="hero">
        <h1 id="title">[Title]</h1>
        <p>[One-sentence instruction]</p>
      </header>
      <main id="main">
        <!-- generated by JS from data array -->
        <div class="controls">
          <!-- Reset, Expand all, Print to PDF -->
        </div>
      </main>
    </div>
  </section>
  <script>
    (function () {
      const items = [ /* data array */ ];
      // render
      // event handlers
      // keyboard nav
      // print handling
    }());
  </script>
</body>
</html>
```

When in doubt

If you are about to write a second copy of anything (HTML block, CSS rule, content paragraph), stop. There is almost always a way to define it once and reference it. The single biggest time sink in this codebase is duplication that has drifted out of sync over time.

If you are not sure whether something is accessible, default to using a real semantic element (`<button>`, `<main>`, `<nav>`, `<section>`, `<h1>`–`<h3>`). The browser does the work for you.

If a piece of source content from faculty looks wrong (a typo, a confused diagram, a numbering scheme that does not match the text), flag it before you build around it. Building a beautiful interactive on top of broken source content just preserves the problem.